PROJECT NEXUS — SYNCED N

COMPREHENSIVE TECHNICAL DEFENSE & SYSTEM ARCHITECTURE MANUAL

Deep-Dive for Android Studio, Cloud Firestore, Brevo, Cloudinary & MIT Panelist Critiques

Academic Year 2026–2027

SECTION 1: HIGH-LEVEL ARCHITECTURE & REAL REPOSITORY STRUCTURE

Ang **Synced N: Church Music Ministry Schedule & Lineup Management System** ay isang native Android application na binuo gamit ang **Kotlin 2.0+ (compileSdk: 37, targetSdk: 37, minSdk: 24)**. Ang architecture nito ay nakabatay sa **Model-View-Controller (MVC)** na may **Repository Pattern** upang maging hiwalay ang UI layer sa backend business logic at cloud services.

Repository Package & Core File Mapping:

- **Application ID:** `com.example.syncedn (Min SDK: 24 Android 7.0, Target SDK: 37)`
- **Frontend Activities:** `MainActivity.kt (Auth/OTP), AdminActivity.kt (Leader 7,936 lines), MemberActivity.kt (Member 3,500+ lines)`
- **Backend Engine:** `data/database/TeamRepository.kt (4,913 lines) & FirestoreManager.kt (634 lines)`
- **Auth & Email OTP:** `data/auth/AuthManager.kt & EmailOtpService.kt (Brevo REST API v3)`
- **Cloud Media Storage:** `data/storage/CloudinaryUploader.kt & ImageLoader.kt (Client-side 512px downscaling)`
- **Security Rules:** `firestore.rules (694 lines ng production-grade RBAC validators)`
- **Stage Usability:** `ChordSheet.java, ChordSheetInput.kt, MetronomePcm.java, StableScrollView.kt`

SECTION 2: UML ARCHITECT & TECHNICAL WRITER (DIZON, JESSA MAE G.)

Eksaktong Paliwanag sa Bawat Isa sa Apat (4) na Diagram sa Chapter 3:

Diagram 1: Use Case Diagram (Section 3.4.1, Page 27)

Paano Ipapaliwanag sa Panelist (Verbatim Script):

> *"Ipinapakita sa Use Case Diagram ang hangganan ng system (System Boundary) at ang dalawang pangunahing aktor: ang Ministry Leader (Admin) at ang Ministry Member (Musicians/Singers). 1. Role Segregation: Ang Leader ang may administrative use cases tulad ng 'Manage Teams', 'Manage Services', 'Assign Members & Roles', at 'Create/Manage Song Lineups'. 2. Relationship Stereotypes (<> vs <>): Ang paggawa ng service schedule ay may <> relationship sa 'View Schedule' at 'View Song Lineup' dahil bawat schedule ay awtomatikong binubuo ng specific service details at song setlist. Samantala, ang 'Request Substitution' at 'Accept/Decline Substitution' ay may <> relationship sa 'View Record/History' dahil nag-ti-trigger lang ang substitution flow kapag may emergency conflict o hindi makakarating ang miyembro. 3. Member Functions: Ang mga miyembro ay may nakalaang use cases para sa 'Update Availability', 'Request Inclusion/Exclusion', 'Access Song Library', at 'Participate in Group Conversation'."*

Key Architectural Concepts na Dapat I-Highlight:

- Actor: Ministry Leader (Primary Initiator para sa Service at Roster Lifecycle)
- Actor: Ministry Member (Volunteer Musician, nagmamarka ng availability at tumatanggap ng assignments)
- Boundary: Synced N Android App na nag-uugnay ng real-time state mula sa Cloud Firestore

Diagram 2: Activity Diagrams (Section 3.4.2, Page 28)

Paano Ipapaliwanag sa Panelist (Verbatim Script):

"Ang Activity Diagrams ng Synced N ay nahahati sa 3 Swimlanes: Ministry Leader, Synced N System (Engine), at Ministry Member. Ito ay nagpapakita ng deterministic logic flow ng aming core workflows: 1. Authentication & Team Access Flow: Parehong dadaan sa Login/Registration form. Ibe-verify ng System ang credentials sa Firebase Auth. Kapag verified, magba-branch ang system: Leader Dashboard o Member Dashboard batay sa 'role' field sa /users/{uid}. 2. Conflict Detection & Substitution Flow: Kapag nag-set ng schedule ang Leader o nag-file ng unavailability ang Member, dadaan ito sa Decision Node: 'Any conflicts or substitution needed?'. Kung 'Yes', mag-a-alert ang system at papayagan ang substitution evaluation; kung 'No', diretso itong mag-po-process at mag-u-update sa Firestore. 3. Realtime Push & Sync Flow: Pagkatapos ma-save ang changes sa Cloud Firestore, magdi-dispatch ang system ng background updates patungo sa Member screen upang ma-display ang bagong lineups at rehearsal details nang sabay-sabay."

Key Architectural Concepts na Dapat I-Highlight:

- 3 Swimlanes: Leader (Manual decisions), Synced N (Algorithmic checks), Member (Response/View)
- Decision Diamonds: May conditional branching para sa invalid login, role conflict, at substitution approval
- Terminal State: End Session na nag-te-terminate sa token session

Diagram 3: Class Diagram (Section 3.4.3, Page 29)

Paano Ipapaliwanag sa Panelist (Verbatim Script):

"Ang aming Class Diagram ay nagpapatunay ng matibay na Object-Oriented Design (OOD) at Model-View-Controller structure ng application: 1. Generalization / Inheritance: Ang base class ay 'User' (naglalaman ng id, name, email, password, role, isMember, createdAt). Mula rito, nag-i-inherit ang 'Ministry Leader' (may leaderId, position, at administrative methods tulad ng manageServices(), assignMembers()) at 'Ministry Member' (may memberId, availabilityStatus, at requestSubstitution()). 2. Multiplicity & Associations: Ang isang Team ay may 1-to-many (1..) na TeamMember. Ang isang Ministry Leader ay pwedeng mag-schedule ng 0..* na Service. Bawat Service ay may 0..* na ServiceAssignment at 1 SongLineup. Ang isang SongLineup ay naglalaman ng 0..* na Song entities. 3. Transactional Classes: Ipinapakita rin dito ang 'Availability' (date, status, notes), 'SubstitutionRequest' (requesterUid, substituteUid, status), at 'Notification'/'Message' classes na may kani-kanilang encapsulation at operation methods."*

Key Architectural Concepts na Dapat I-Highlight:

- Base Class: User (Id, Name, Email, Role) na ini-extend ng Leader at Member
- Multiplicity: 1 Team -> 1..* TeamMember; 1 Service -> 0..* Assignment -> 1 SongLineup
- Data Encapsulation: Lahat ng properties ay private/immutable na may explicit getters at business operations

Diagram 4: Entity-Relationship Diagram - ERD (Section 3.4.4, Page 30)

Paano Ipapaliwanag sa Panelist (Verbatim Script):

"Bagamat NoSQL Document Store ang Cloud Firestore, ang logical ERD ang aming mathematical blueprint upang masigurong walang data anomalies at may strict referential integrity: 1. Primary Keys (PK) & Foreign Keys (FK): Ang table na USER ay may PK na user_id (matches Firebase Auth UID). Ito ay nag-re-reference sa MEMBER_ROLE gamit ang member_id bilang FK. Ang SERVICE table ay may PK na service_id na nagsisilbing foreign key sa ASSIGNMENT table (FK: service_id, FK: member_id, FK: role_id) at LINEUP_ITEM table (FK: service_id, FK: song_id). 2. Normalization & Many-to-Many Resolution: Ang relasyon sa pagitan ng MEMBER at ROLE ay na-normalize gamit ang bridge entity na MEMBER_ROLE. Ganito rin sa SERVICE at SONG na pinag-uugnay ng LINEUP_ITEM na nagtataglay ng specific song_key at song_order para sa service na iyon. 3. NoSQL Physical Mapping: Sa aming actual implementation sa Firestore, ang mga relational FK entities na ito ay na-optimize bilang subcollections (e.g. /teams/{teamCode}/services/{serviceId}) para sa high-speed querying at zero-join retrieval."

Key Architectural Concepts na Dapat I-Highlight:

- Bridge Entities: MEMBER_ROLE, ASSIGNMENT, at LINEUP_ITEM upang i-resolve ang M:N relationships
- Referential Integrity: Bawat service record ay may explicit binding sa valid member at song ID
- Schema Normalization: 3NF logical structure bago i-denormalize para sa NoSQL Firestore reads

SECTION 3: DATABASE ADMINISTRATOR (PENDRE, JUSTIN KYLE D.)

Mula sa Setup sa Firebase Console hanggang sa 694 lines ng firestore.rules:

Step-by-Step Implementation sa Firebase Console:

1. **Project Provisioning:** Lumikha ng bagong Google Firebase Project na pinangalanang 'Synced N' sa Google Cloud Console.
2. **Authentication Setup:** In-enable ang 'Email/Password' Sign-in method at 'Google Sign-In' gamit ang OAuth 2.0 Web Client ID (74207597857-f0c167eg3u06hdvu03h3671ftgt9q8dj.apps.googleusercontent.com).
3. **Firestore Database Creation:** In-initialize ang Cloud Firestore sa **Production Mode** (default deny-all). Pinili ang rehiyon na pinakamalapit sa Pilipinas (asia-southeast1 / Singapore) upang matiyak ang mababang network latency (<80ms).
4. **Android App Registration & Connection:** Ni-register ang package name com.example.syncedn. Kinuha ang SHA-1 at SHA-256 digital fingerprint signatures mula sa Gradle Signing Report (./gradlew signingReport) at in-upload sa Firebase Project Settings.
5. **Configuration File Insertion:** I-dinownload ang nabuong google-services.json at inilagay sa folder na c:\Users\RYAN\AndroidStudioProjects\Synced1.0\app\google-services.json.
6. **Gradle Dependencies:** Sa app/build.gradle.kts, inilagay ang alias(libs.plugins.google.services) at in-import ang official Firebase BoM (Bill of Materials) kasama ang libs.firebase.auth at libs.firebase.firestore.

Database Schema & Document Structure (Eksaktong Collections):

• **Root Collection /users/{userId}:** Document ID = Firebase Auth UID. May fields na uid, username, email, role ('Leader' o 'Member'), teamCode (6 chars), ministryRole, unavailableRanges (List ng dates), photoUri (Cloudinary HTTPS URL), at removedByLeader (boolean).
• **Root Collection /teams/{teamCode}:** Document ID = 6-character uppercase code (e.g. 'W4K9LM'). Naglalaman ng leaderUid, leaderName, roles, at removedMembers.
• **Subcollection /teams/{teamCode}/services/{serviceId}:** Naglalaman ng title, date, start, end, venue, status ('Draft'/'Published'), assignments (List ng Maps ng roles at member UIDs), at songs.
• **Subcollection /teams/{teamCode}/songs/{songId}:** Naglalaman ng title, artist, key, bpm, sheet (Chordpro formatted text), lyrics, at referenceUrl.
• **Subcollection /teams/{teamCode}/announcements/{announcementId}:** Priority ministry broadcasts na may urgent boolean flag.
• **Subcollection /teams/{teamCode}/substitutions/{substitutionId}:** Handshake logs: requesterUid, substituteUid, role, reason, at status ('Pending', 'Accepted', 'Approved', 'Declined').
• **Subcollection /teams/{teamCode}/messages/{messageId}:** Real-time team chat na may senderUid, text, mediaUrl, timestamp, at isDeleted flags.

Deep-Dive sa firestore.rules (694 Lines ng Security Validation):

Ang aming Firestore Security Rules ay **HINDI** naka-test mode kundi production-grade security architecture:
1. **Default Deny Rule (Line 537):** match /{document=**} { allow read, write: if false; } — Walang sinuman ang makaka-access sa database nang walang explicit rule.
2. **Authentication & Ownership (Lines 167-188):** Lahat ng operation ay kailangang dumaan sa isAuthenticated() (request.auth != null) at isDocOwner() para hindi ma-overwrite ng isang user ang profile ng ibang user.
3. **Multi-Tenant Isolation (Line 282):** Function isSameTeam(teamCode) kung saan bina-validate na ang teamCode ng requester ay kapareho ng teamCode ng document. Bawal mabasa ng ibang church ang data ng ibang church.
4. **Role-Based Access Control (RBAC - Line 271):** Ang function na isLeaderOfTeam(teamCode) ay nagbabasa sa root /teams/{teamCode} upang patunayan na request.auth.uid == team.leaderUid. Tanging ang Leader lang ang pinapayagang mag-write sa services at announcements.
5. **Field-Level Validation & Sanitization:** May functions tulad ng hasOnlyAllowedFields(), validStringLength(min, max), isValidEmail(), at regex enforcement para sa 6-character team codes (^[A-Z0-9]{6}$).

SECTION 4: DEVOPS & DEPLOYMENT SPECIALIST (LAGARE, JOSHUA A.)

Eksaktong Proseso ng Pag-Build, Pag-Sign, at Pag-Deploy ng Android APK:

1. Build Pipeline & Gradle Tasks:
• Ginamit ang **Android Gradle Plugin (AGP 8.x+)** kasama ang Kotlin Gradle Plugin. Ang buong project compilation ay pinapatakbo sa pamamagitan ng Gradle Wrapper (./gradlew).
• Para sa development at testing, binuo ang Debug APK gamit ang command na: ./gradlew assembleDebug na naglalabas ng artifact sa app/build/outputs/apk/debug/app-debug.apk.
• Para sa production packaging, ginamit ang command na: ./gradlew assembleRelease.

2. Keystore Signing & Security:
• Gumamit ng Java Keytool upang mag-generate ng 2048-bit RSA cryptographic signing key: keytool -genkey -v -keystore syncedn.jks -keyalg RSA -keysize 2048 -validity 10000 -alias syncedn.
• Sa app/build.gradle.kts (Lines 52-59), na-configure ang buildTypes { release { signingConfig = signingConfigs.getByName("debug") } } para sa seamless evaluation testing nang hindi nanganganib ma-expose ang release production credentials sa public git.

3. Environment Secrets Management:
• Mahigpit na ipinagbawal ang pag-hardcode ng API keys sa repository. Sa build.gradle.kts (Lines 9-24), lumikha ng custom resolver na resolveEnvSecret() na nagbabasa mula sa local.properties at .env file.

• Ang mga secrets na BREVO_API_KEY, BREVO_SENDER_EMAIL, CLOUDINARY_CLOUD_NAME, at CLOUDINARY_UPLOAD_PRESET ay awtomatikong ini-inject bilang dynamic BuildConfig fields sa runtime.

4. Sideloading, Internal Testing & Deployment Distribution:
• Para sa UAT sa New Life Christian Church, ang distribution ay isinagawa gamit ang **Firebase App Distribution** at direktang **USB Sideloading via ADB** (adb install -r SyncedN.apk). Tiniyak na naka-enable ang 'Install Unknown Apps' permission sa mga Android test devices.

SECTION 5: LEAD BACKEND DEVELOPER (VILLAR, JOHN LLOYD F.)

Saan Matatagpuan ang Backend sa Source Code at Paano Tumatakbo ang Business Logic:

1. Lokasyon ng Backend Engine sa Codebase:
• Ang buong backend business logic ay **HINDI** nakakalat sa UI activities. Ito ay naka-encapsulate sa dalawang pangunahing klase:
a. app/src/main/java/com/example/syncedn/data/database/TeamRepository.kt (4,913 lines) — Core Data Access & State Orchestrator.
b. app/src/main/java/com/example/syncedn/data/database/FirestoreManager.kt (634 lines) — Low-level Firestore CRUD & Session Caching.

2. In-Memory Reactive Caching Engine (Zero Latency on Stage):
• Sa TeamRepository.kt (Lines 38-66), nag-implement ako ng multi-layered in-memory cache gamit ang java.util.concurrent.ConcurrentHashMap:
- cachedRosters, cachedServices, cachedSongs, cachedAnnouncements, cachedSubstitutions.
• *Bakit ito ginawa?* Sa loob ng simbahan, madalas na mahina ang mobile data o Wi-Fi. Sa pamamagitan ng memory cache at SharedPreferences sync, 0-milliseconds ang response time ng paglipat ng tabs sa screen at hindi mag-ka-crash ang app kung mawalan ng koneksyon habang nagpapatugtog.

3. Conflict Detection Algorithm:
• Bago i-save ang isang bagong service schedule, tinatawag ang verification engine. Sinusuri nito ang bawat miyembro sa dalawang aspeto:
1. *Availability Calendar Match:* Bina-validate kung ang petsa ng service ay tumatama sa unavailableRanges ng user.
2. *Double-Booking Check:* Bina-validate kung ang miyembro ay naka-assign na sa isa pang service na may overlapping start at end time.
• Kapag may conflict, nagbabalik ang backend ng structured warning record kaya napipigilan ang leader bago pa ma-publish ang schedule.

4. Substitution Handshake State Machine:
• Makikita sa TeamRepository.kt (Line 3670: createSubstitution() at Line 4033: acceptSubstitution()).
• Gumagamit ito ng deterministic status machine: Pending ➔ Accepted ➔ Approved.
• Sa sandaling i-click ng substitute o ng leader ang 'Accept', awtomatikong binubuksan ng backend ang linked ServiceData document sa Firestore, hinahanap ang lumang assignee sa assignments list, pinapalitan ito ng bagong substitute UID, at sine-save ang update gamit ang atomic transaction.

SECTION 6: CLOUD INTEGRATIONS (BREVO REST API & CLOUDINARY CDN)

1. Brevo REST API v3 — Email OTP & Account Verification:
• **File:** app/src/main/java/com/example/syncedn/data/auth/EmailOtpService.kt (271 lines)
• **Bakit hindi Firebase Default Reset Email?** Ang generic Firebase email ay madalas pumasok sa Spam folder o ma-block ng email security filters. Sa pamamagitan ng Brevo Transactional SMTP REST API (https://api.brevo.com/v3/smtp/email), nakakapagpadala kami ng customized, professional HTML 6-digit verification code.
• **Cryptographic Security:** Gumamit ng java.security.SecureRandom para sa unpredictable 6-digit generation. May strict parameters: OTP_VALIDITY_MS = 10 minutes, RESEND_COOLDOWN_MS = 60 seconds, at MAX_OTP_ATTEMPTS = 5 upang maiwasan ang brute-force attacks.

2. Cloudinary CDN — Client-Side Optimized Image Pipeline:
• **File:** app/src/main/java/com/example/syncedn/data/storage/CloudinaryUploader.kt (288 lines)
• **Bakit Cloudinary?** Ang Cloudinary ay nag-aalok ng high-speed global image delivery network nang libre nang walang strict egress bandwidth caps ng Firebase Storage.
• **Client-Side Image Compression:** Bago i-upload ang raw camera photo (na karaniwang 5MB–10MB), binabasa ng device ang EXIF orientation (ExifInterface), dina-downscale ang imahe sa maxDimension = 512px, at kino-compress sa 85% JPEG quality. Dahil dito, ang file size na ini-upload ay nagiging mas mababa sa 150KB—napakabilis kahit mahina ang signal ng phone.

SECTION 7: MASTER IN INFORMATION TECHNOLOGY (MIT) PANELIST CRITIQUES & DEFENSE ANSWERS

Kung ako ang MIT Panelist, narito ang mga pinakamabigat na teknikal na tanong, butas, at kahinaan na aking sisitahin, kasama ang inyong bala at opisyal na depensa:

MIT Question 1: 'Sabi ninyo MVC ang architecture, pero bakit ang AdminActivity.kt ninyo ay umabot ng halos 8,000 lines? Hindi ba ito Anti-Pattern na tinatawag na Massive View Controller (Fat Activity)?'

Opisyal na Depensa ng Grupo (Verbatim Script):

> *"Opo, Panelist, kinikilala po namin na mataas ang line count ng aming AdminActivity.kt. Ngunit kung susuriin po ang aming software architecture, hindi ito monolithic spaghettified code. 1. Separation of Concerns: Ang lahat ng persistent state, network calls, at Firestore business logic ay mahigpit na naka-isolate sa TeamRepository.kt at FirestoreManager.kt. Walang direct raw database querying na nakasulat sa loob ng UI buttons. 2. Complex Stage Domain: Ang AdminActivity ay nagho-host ng 8 interconnected ministry management modals (Service Creation Wizard, In-App Chord Transposition Engine, YouTube Live Previewer, at Multi-user Roster Manager) na nangangailangan ng masinsinang lifecycle listeners at hardware orientation guards. 3. Future Roadmap: Bilang pagpapabuti alinsunod sa Chapter 5 Recommendations, aming plano na i-refactor ang AdminActivity patungo sa modernong MVVM (Model-View-ViewModel) gamit ang Jetpack Compose Navigation upang mahati ito sa maliliit na screen composables."*

Core Defense Strategy: Depensa: I-highlight na nasa TeamRepository ang business logic, at aminin na recommendations ito for MVVM refactoring.

MIT Question 2: 'Bakit kayo gumamit ng NoSQL Cloud Firestore gayong napakalinaw na Relational ang inyong data (User, Team, Service, Lineup, Song)? Hindi ba magastos sa read/write billing kapag lumaki ang data?'

Opisyal na Depensa ng Grupo (Verbatim Script):

"Magandang punto po, Panelist. Sinadya po namin ang pagpili sa Cloud Firestore dahil sa 3 pangunahing bentahe para sa aming specific church domain: 1. Native Real-Time Synchronization (WebSockets/Snapshot Listeners): Sa isang church service, kapag nagpalit ng kanta o key ang worship leader habang nagre-rehearse, kailangang mag-reflect agad ito sa screen ng mga gitarista at piyanista nang walang manual screen refresh. Ang Relational DB tulad ng PostgreSQL o MySQL ay mangangailangan ng kumplikadong external WebSocket server o frequent polling. 2. Built-in Offline Persistence: Ang Cloud Firestore ay may automatic local SQLite/cache storage sa Android. Kapag nawalan ng internet sa loob ng simbahan, patuloy na mababasa ng musicians ang naunang na-load na chord sheets at schedule. 3. Subcollection Scoping: Sa pamamagitan ng pag-organisa ng data sa ilalim ng /teams/{teamCode} subcollections, napanatili naming mababa ang document read costs dahil scoped lamang ang queries sa aktibong team."

Core Defense Strategy: Depensa: Bigyang-diin ang Real-time Snapshot Listeners at Offline Caching na hindi libre sa traditional Relational SQL.

MIT Question 3: 'Wala ba kayong Race Condition o Concurrency Issue kapag sabay na nag-accept ang dalawang substitute sa iisang service role?'

Opisyal na Depensa ng Grupo (Verbatim Script):

"Naiwasan po namin ang concurrency issue na ito sa pamamagitan ng aming Handshake Logic sa TeamRepository.kt at Firestore Security Rules. 1. Atomic Evaluation: Ang bawat substitution document sa /substitutions/{subId} ay may strict initial state na 'Pending'. Kapag ang unang substitute ay nag-click ng 'Accept', ang status ay agad na nag-a-update sa 'Accepted' kasabay ng pag-record ng kanyang specific substituteUid. 2. Conditional Rules Validation: Sa aming firestore.rules, may kondisyon na nagbabawal mag-update sa isang substitution document kung ang kasalukuyang status nito ay hindi na 'Pending'. Dahil dito, ang anumang kasunod na request ay ire-reject ng Firestore engine bago pa man ito makapasok sa database."

Core Defense Strategy: Depensa: I-cite ang Firestore Security Rules status machine check at atomic state transition.

MIT Question 4: 'Nakalagay sa local.properties ang API keys, pero paano ninyo pinoprotektahan ang Brevo at Cloudinary credentials sa APK decompilation gamit ang reverse-engineering tools tulad ng JADX?'

Opisyal na Depensa ng Grupo (Verbatim Script):

"Salamat po sa napakahalagang security question, Panelist. 1. Build-Time Injection: Ang mga secrets ay hindi naka-hardcode sa git source code kundi dynamic na binabasa mula sa environment variables at local.properties bago i-compile ng Gradle sa BuildConfig. 2. Cloudinary Unsigned Upload Preset: Ang aming Cloudinary configuration ay gumagamit ng 'Unsigned Upload' preset (synced_preset) na may restricted write-only capability (walang permission mag-delete o magbasa ng ibang files sa Cloudinary account). 3. Fallback Obfuscation: Sa EmailOtpService.kt, nag-implement kami ng FALLBACK_MASKED_BYTES na ginamitan ng XOR-byte masking (xor 0x5A) upang hindi madaling mabasa sa static string analysis kapag na-decompile ang APK."

Core Defense Strategy: Depensa: Ipaliwanag ang XOR Byte Masking sa EmailOtpService.kt at Unsigned Write-Only Presets.

MIT Question 5: 'Bakit Android-Only at walang iOS o Web app? Paano kung iPhone ang gamit ng Pastor o Worship Leader?'

Opisyal na Depensa ng Grupo (Verbatim Script):

> *"Alinsunod po ito sa aming Scope and Limitations sa Chapter 1 (Section 1.4, Limitations #1 at #9). 1. Beneficiary Demographic: Sa isinagawang preliminary survey at interview kay Brother Nathaniel Ceralvo ng New Life Christian Church, 100% ng kanilang 15 active music ministry volunteers at leaders ay gumagamit ng Android smartphones. 2. Native Hardware Optimization: Ang pag-develop gamit ang Native Android (Kotlin) ay nagbigay-daan upang makabuo kami ng low-latency audio engine para sa Metronome (MetronomePcm.java) at smooth 60fps auto-scrolling sa chord sheets na mahirap maabot sa hybrid web-wrapper apps sa mababang budget. 3. Cross-Platform Extensibility: Nakasaad po sa aming Chapter 5 Recommendations na ang modular NoSQL structure sa Cloud Firestore ay 100% handa nang kabitan ng Flutter o React Web dashboard para sa future expansion."*

Core Defense Strategy: Depensa: I-angkla sa actual church interview data at native audio/scroll hardware performance.

SECTION 8: 10 TEAM MEMBERS RESPONSIBILITY MATRIX (FOR GC COPY-PASTE)

Member & Role	Primary Technical Scope	Defense Focus / Deliverable
Genturo, Ryan Lloyd Project Manager	Agile Sprints, Bug Triage, Unit Testing, App Deployment	Project Timeline, Bug Resolution metrics, App Packaging
Dizon, Jessa Mae UML Architect / Writer	Use Case, Activity, Class, ERD, Academic Cohesion	Diagram stereotypes (include/extend), 3NF Normalization
Gregorio, Ralph Lauren UI/UX Lead Designer	Figma Design System, Contrast (#0D1B2A), Thumb-zone	Stage visibility ergonomics, low-glare dark palette
Lagare, Joshua DevOps / Product Owner	Git branching, AGP 8.x, ProGuard/R8, Church Domain	Build pipeline, APK signing, Ministry user needs
Lebuna, Angelo Daniel QA Engineer	Integration Testing, Edge Case Validation, Bug Log	Conflict alert accuracy, offline recovery, stress tests
Pendre, Justin Kyle Database Administrator	Cloud Firestore, 694-line firestore.rules, Schema	NoSQL subcollections, RBAC security, zero data leaks
Ramirez, John Michael System Analyst	SOP 1-5 to Feature Mapping, Process Evaluation	Demonstrating how Synced N solved NLCC's problems
Salva, Ma. Paula UAT Coord. / Pitch	ISO/IEC 25010 Surveys, NLCC User Feedback, Presentation	Usability & Acceptability ratings (>4.5/5.0 Excellent)
Sarmiento, Mark Allen Lead Frontend Developer	Material 3 UI, ChordSheetInput, MetronomePcm, Scroll	Stage tooling implementation, custom views, smooth UI
Villar, John Lloyd Lead Backend Developer	TeamRepository (4.9k lines), Cache, Conflict Engine	In-memory caching, Realtime listeners, Substitution logic